

PRIMED: Preliminary Investigation of Meta-analytic Databases

A workflow function for exploratory analysis of dependent effect sizes

Your Name

2026-04-29

Table of contents

1	Introduction	2
2	Why a workflow at all?	2
3	Loading the function	3
4	A worked example	3
5	Running the workflow	4
6	Step 1: Data and dependence structure	4
7	Step 2: Moderators	5
7.1	Categorical moderators	5
7.2	Continuous moderators	5
7.3	Missingness table	6
7.4	Hierarchical structure	6
8	Step 3: Standard errors and weights	6
8.1	Standard errors by sample	6
8.2	Scaled SEs (SMDs only)	7
8.3	ISC weights	7
9	Step 4: Effect size distribution	8
9.1	Marginal density	8
9.2	Hierarchical forest plot	8
9.3	Funnel plots	9

10 Cycling back before modelling	9
11 Customisation	9
12 Reporting from the workflow	10
13 Summary	10
References	11

1 Introduction

Meta-analyses of dependent effect sizes routinely involve complex hierarchical data — multiple effect sizes nested within samples, samples nested within studies, moderator variables at several levels, and auxiliary statistics like standard errors that behave differently across effect size metrics. Before fitting any formal model, a careful analyst needs to understand what’s actually in the database.

The **PRIMED workflow** (PReliminary Investigation of MEta-analytic Databases) proposed by Pustejovsky, Zhang, and Tipton (2026) organises this preliminary work into four sequential steps:

1. Describe the amount of data and dependence structure
2. Explore study characteristics and potential moderators
3. Inspect standard errors and other auxiliary data
4. Visualise the distribution of effect size estimates

Crucially, the effect size distribution is examined **last**, which preserves some room to refine modelling decisions without being influenced by the results themselves.

This vignette demonstrates the `primed()` function, which automates the workflow — producing the summary tables and visualisations for each step in a single call.

2 Why a workflow at all?

Experienced meta-analysts already do preliminary checks, but the process is rarely written down or reported. As databases grow larger and more complex, informal inspection becomes infeasible. A structured workflow helps by:

- Verifying database integrity before modelling
- Building understanding of between- vs within-study variation
- Informing decisions about model structure (e.g., whether three-level models are feasible)
- Identifying sparse categories, outliers, and missingness problems early

- Separating data-exploration decisions from outcome-driven choices, reducing opportunities for HARKing

The `primed()` function is an opinionated implementation that produces a reasonable default set of outputs. It's designed as a starting point that analysts can extend or customise.

3 Loading the function

```
source("primed.R")
```

Dependencies: `dplyr`, `tidyr`, `ggplot2`, `purrr`, `tibble`, `rlang`, `forcats`.

4 A worked example

We'll use the `dat.assink2016` dataset from the **metafor** package, which contains 100 effect sizes from 17 studies examining the effect of various psychotherapies on recidivism in juvenile delinquents. Each study contributes multiple effect sizes — a classic dependent-effects structure.

```
library(metafor)

data("dat.assink2016", package = "metafor")

dplyr::glimpse(dat.assink2016)
```

The key variables are:

- `study` — study identifier
- `esid` — effect size identifier within study
- `yi` — effect size estimate (standardised mean difference)
- `vi` — sampling variance (we'll convert to SE)
- `delttype` — outcome type (a potential moderator)

```
meta_df <- dat.assink2016 |>
  dplyr::mutate(
    se = sqrt(vi),
    df = 100 # approximate - dataset doesn't provide exact df
  )
```

5 Running the workflow

A single call runs all four steps:

```
results <- primed(
  data      = meta_df,
  es_col    = "yi",
  se_col    = "se",
  study_col = "study",
  sample_col = NULL,          # one sample per study here
  moderators = c("deltype", "year"),
  es_type    = "SMD",
  df_col     = "df",
  rho_values = c(0.3, 0.5, 0.7)
)

print(results)
```

The returned object is a list with three slots:

- `results$summary` — high-level counts and descriptive stats
- `results$plots` — a named list of `ggplot` objects
- `results$tables` — a named list of summary tibbles

6 Step 1: Data and dependence structure

The first step counts observations at each level and visualises how effects are nested within higher-level units.

```
# Basic structure summary
results$tables$step1_structure

# How many effect sizes contribute to each study/sample?
results$plots$step1_es_per_sample
```

What to look for:

- Is the total number of studies consistent with your inclusion criteria?
- How much variation is there in the number of effects per study? A skewed distribution (a handful of studies contributing 20+ effects while most contribute 1–2) signals that meta-analytic estimates may be sensitive to modelling assumptions about dependency.

- Are there studies with implausibly many effects? That’s often a coding error.

If `sample_col` is supplied (i.e., studies contribute multiple participant samples), a second plot summarises samples per study — useful for deciding whether a three-level model (effects within samples within studies) is feasible.

7 Step 2: Moderators

Step 2 examines every moderator you list. For each one, the function decides whether to treat it as categorical or continuous and produces appropriate summaries.

7.1 Categorical moderators

Categorical moderators get paired bar charts showing the number of studies and number of effects per level:

```
results$plots$step2_cat_deltypes
```

What to look for:

- **Typos and inconsistent coding** — “Control”, “control”, “CONTROL” will appear as separate levels
- **Sparse categories** — fewer than 3–5 studies in a level means moderator tests will be underpowered and estimates unstable
- **Missingness** — a “Not Reported” bar signals how often the variable is unavailable
- **Study vs effect counts** — for effect-level moderators, the effect count exceeding the study count indicates within-study variation (which you can exploit in meta-regression)

7.2 Continuous moderators

Continuous moderators get density plots with rug marks showing individual observations:

```
results$plots$step2_cont_year
```

What to look for:

- Bimodal distributions (maybe the variable is really categorical in disguise)
- Outliers that could be influential
- Narrow range — a moderator that barely varies can’t explain effect heterogeneity

7.3 Missingness table

```
results$tables$step2_missingness
```

This lets you see at a glance which moderators have enough data to include in a meta-regression and which are too sparse to be useful.

7.4 Hierarchical structure

For categorical moderators, a co-occurrence table shows how often pairs of categories appear together *within the same study*:

```
results$tables$step2_cooccurrence$cooccurrence_deltypes
```

Diagonal cells report the total number of studies (and effects) for each level. Off-diagonal cells report co-occurrences — studies that include both levels. A table with empty off-diagonals tells you the moderator has *no within-study variation*; all its explanatory power comes from between-study differences, which are confounded with everything else that differs across studies.

For continuous moderators, a decomposition table separates the variable into between-sample and within-sample components:

```
results$tables$step2_decomposition$decomposition_year
```

If the “Sample-Mean-Centered” row has substantial spread, the moderator varies within samples and can be used to estimate within-sample effects. If it’s nearly zero, the moderator is effectively sample-level and you’re stuck with the between-sample interpretation.

8 Step 3: Standard errors and weights

Step 3 focuses on the auxiliary statistics that drive modelling but aren’t the primary quantity of interest.

8.1 Standard errors by sample

```
results$plots$step3_se_by_sample
```

Each row is a sample; each point is the SE of an individual effect. For SMDs and Fisher-z correlations, SEs from the same sample should be *very similar* (they depend mostly on sample size). Rows with wide scatter warrant investigation — the paper notes this often happens when:

- Studies with more than two treatment arms have different n in each arm
- Follow-ups had different numbers of respondents
- Outcomes had different amounts of missing data

8.2 Scaled SEs (SMDs only)

The SE of a standardised mean difference depends on the SMD itself, which induces artefactual variation even when the underlying n is identical. The scaled SE removes this artefact:

$$SE_{\text{scaled}} = \sqrt{SE^2 - \frac{d^2}{2\nu}}$$

```
results$plots$step3_scaled_se
```

Scaled SEs should be nearly constant within a sample. Variation here has the same diagnostic implications as above, minus the artefact from the effect size magnitude.

8.3 ISC weights

The **inverse sampling covariance weights** show how much each sample contributes to the overall average under a simplified model, as a function of the assumed within-sample correlation ρ :

```
results$plots$step3_isc_weights
```

What to look for:

- A few samples dominating the weight distribution — your results will be sensitive to including or excluding them
- Weights that shift substantially with ρ — your results will be sensitive to the assumed correlation structure

If one or two studies soak up 25%+ of the weight, plan on reporting sensitivity analyses that drop them.

9 Step 4: Effect size distribution

Only now, having validated everything else, do we look at the effect sizes themselves.

9.1 Marginal density

```
results$plots$step4_es_density  
results$plots$step4_sample_es_density
```

Solid vertical lines mark the quartiles; dashed lines mark Tukey fences (by default, 3 times the IQR beyond the quartiles — you can change this with the `fence_multiplier` argument). Points outside the fences are flagged as potential outliers.

The sample-level density (averaged across effects within each sample) is typically tighter than the effect-level one. The gap between them is the *within-sample* heterogeneity.

```
results$tables$step4_outliers_effect  
results$tables$step4_outliers_sample
```

What to look for: outliers are flags for *investigation*, not automatic removal. Common causes are coding errors (decimal misplacement, wrong sign), unit mismatches (correlation reported where SMD expected), and genuinely influential studies that are important to report on.

9.2 Hierarchical forest plot

```
results$plots$step4_forest
```

Unlike a conventional forest plot, effects are grouped on a single row per sample rather than one-per-row. Samples are sorted by their average effect. This view lets you see:

- **Marginal outliers** — points far from the overall distribution
- **Within-sample outliers** — points that disagree with their own sample's other estimates
- **Heterogeneity sources** — whether spread is mostly between samples or within samples

The answer to that last question informs random-effects structure. Heavy between-sample spread + tight within-sample spread → a two-level model with sample-level random effects is probably fine. Heavy spread at both levels → you want a three-level model or robust variance estimation.

9.3 Funnel plots

```
results$plots$step4_funnel      # effect-level  
results$plots$step4_funnel_sample # aggregated to sample level
```

Asymmetry in the funnel plot is a *preliminary* diagnostic for small-study effects or selective reporting. It is not conclusive — the paper notes that apparent asymmetry can also arise from systematic differences between older/smaller and newer/larger studies (e.g., differences in outcome measurement standards). Treat this as a starting point for formal bias assessments, not as the final word.

10 Cycling back before modelling

The paper recommends **cycling through steps 1–3 before looking at the effect sizes in step 4**. In practice:

- Clean up inconsistent coding revealed in step 2
- Collapse sparse categories if necessary
- Decide which moderators have enough data to include
- Decide whether to decompose continuous moderators into within/between components
- Reconsider any pre-registered analysis plan in light of what’s actually feasible

Only then move to step 4 and the formal model. Documenting these decisions — ideally by sharing the PRIMED output alongside the manuscript — makes the modelling choices transparent and reproducible.

11 Customisation

The function is intentionally modest in scope. Common extensions you might want:

- **Network diagrams** for co-occurrence of categorical moderators (the paper shows this for academic dishonesty data)
- **Scatterplot matrices** via `GGally::ggpairs()` for multivariate moderator structure
- **UpSet plots** via `UpSetR` for multivariate missingness
- **Contour-enhanced funnel plots** using `metafor::funnel()`
- **GOSH plots** for leave-one-out sensitivity

Since `primed()` returns raw `ggplot` objects, you can modify any plot with standard `ggplot` syntax:

```
results$plots$step4_forest +  
  ggplot2::labs(title = "My custom title") +  
  ggplot2::theme(plot.title = ggplot2::element_text(face = "bold"))
```

Or pull the underlying tables and build your own figures.

12 Reporting from the workflow

A reasonable starting point for methods sections:

Prior to fitting meta-analytic models, we conducted a preliminary data analysis following the PRIMED workflow (Pustejovsky et al., 2026). We examined the distribution of effect sizes across studies and samples (Step 1), the marginal and hierarchical structure of candidate moderators (Step 2), the distribution of standard errors and inverse sampling covariance weights under assumed within-sample correlations of $\rho = 0.3, 0.5, 0.7$ (Step 3), and the distribution of effect size estimates at both the effect and sample levels (Step 4). Full workflow output is available in the supplementary materials.

Then describe any decisions that were *informed* by the workflow (category collapsing, moderator exclusion, outlier investigation) separately from the pre-registered analysis plan.

13 Summary

The `primed()` function bundles a standard set of preliminary analyses for meta-analyses of dependent effect sizes into a single call, producing the tables and plots recommended in Pustejovsky, Zhang, and Tipton (2026). The intended workflow is:

1. Run `primed()` on your cleaned database
2. Work through steps 1–3, making and documenting any coding/moderator decisions
3. Only then examine step 4 output
4. Fit your formal meta-analytic model
5. Archive the PRIMED output alongside your analysis code

Sharing these outputs makes the pre-modelling phase of meta-analysis visible and reproducible — something the field has historically under-documented.

References

Pustejovsky, J. E., Zhang, J., & Tipton, E. (2026). A Preliminary Data Analysis Workflow for Meta-Analysis of Dependent Effect Sizes.